

# Deployment Instructions

---

## Music Podcast Player

### 1 INTRODUCTION

This document aims to prepare the reader for deployment of the Music Podcast Player, both locally and to Amazon Web Services (AWS). Additionally, it explains how to configure the Continuous Deployment (CD) GitHub Actions to automatically deploy to AWS. Finally, it provides guidance on the configuration of a deployment in each environment.

The architecture of the Music Podcast Player consists of a frontend, two APIs, a PostgreSQL database, and a DynamoDB table. The PostgreSQL database and one of the APIs are kept inside a Virtual Private Cloud (VPC) without outbound internet access. Additionally, it makes requests to RSS feeds hosted by artists and hosting providers, as well as to the PodcastIndex API. For more detailed information on the architecture, see the Architecture documentation. It may also be helpful to read the Environment Configuration documentation before continuing.

### 2 LOCAL DEPLOYMENT

The Music Podcast Player has been designed to be deployed independently of any cloud provider through the use of Docker Compose in order to facilitate local development. However, while the local deployment doesn't depend on cloud infrastructure, it does need to make calls to the PodcastIndex API in order for the search feature to work, which requires authentication configuration.

The credentials for the PodcastIndex API need to be provided to the API service through two environment variables. The `docker-compose.yml` handles this by importing them from an environment file at `secrets/backend.env`, which has the following format.

```
RSS_PLAYER_PODCAST_INDEX_KEY=<api-key>
RSS_PLAYER_PODCAST_INDEX_SECRET=<api-secret>
```

At the time of writing, both the `api-key` and `api-secret` can be obtained from the PodcastIndex API website by creating an account. Note that the `api-secret` can sometimes contain `$`, which must be escaped for Docker by writing two: `$$`.

Following this configuration, you can deploy the Music Podcast Player locally using the following.

```
docker compose up --build
```

The PostgreSQL image will maintain its state in an anonymous volume between deployments. If you would like to reset its state, you can bring down the volumes.

```
docker compose down -v
```

## 3 AWS DEPLOYMENT

In order to as easy as possible to deploy the Music Podcast Player on AWS, we've used the AWS Cloud Development Kit (CDK) to describe the resources in our deployments. That said, the CDK expects the builds for both the frontend and backend to be available at synthesis time. Additionally, in order to keep our deployments environment-independent, some environment specific configuration must be provided in environment variables. Also, non-rotating secrets such as the PodcastIndex API credentials must be manually created in Systems Manager's Parameter Store.

### 3.1 SETUP AWS ENVIRONMENT

On an AWS account that has never been used to deploy from the CDK before needs to be bootstrapped. First, start off by creating a virtual environment using the `requirements.txt` in the `cdk` directory. Then, after configuring AWS credentials for the command line, run the following.

```
cdk bootstrap
```

Before synthesizing the Cloudformation template or deploying, several environment variables need to be set, either in a `.env` or directly.

1. `DEPLOY_ENVIRONMENT`

This is either set to `development` or `production`. The difference is that email verification is disabled in development, and a DNS record is made for the production deployment.

2. `MPP_DOMAIN_NAME`

The domain name, without a subdomain, used in production. As an example, if the app is hosted at `www.mydeployment.com`, this variable would be set to `mydeployment.com`.

3. `MPP_RECORD_NAME`

The subdomain used in production. In the previous example, this would be set to `www`.

4. `MPP_R53_ZONE_ID`

The Route 53 zone ID to use in production. This can be found under "Hosted zones" in the AWS Console after registering a domain.

5. `MPP_NOREPLY_EMAIL`

The email address to use when sending no reply mail in production. Typically in the form `no-reply@<domain>`.

Additionally, it is not possible to create `SecureString` Systems Manager parameters (static secrets) using the CDK. As such, the following `SecureString` parameters must be created manually.

1. `/rss-music-player/jwt/key`

This is the secret key used by the API service to generate JSON Web Tokens (JWTs) for user authentication. You can generate one randomly in Python using the following.

```
import secrets
import base64
token_bytes = secrets.token_bytes(96)
print(base64.b64encode(token_bytes).decode("UTF-8"))
```

2. `/rss-music-player/podcast-index-api/key`  
The PodcastIndex API key obtained from the PodcastIndex.
3. `/rss-music-player/podcast-index-api/secret`  
The PodcastIndex API secret obtained from the PodcastIndex.

### 3.2 BUILD THE MUSIC PODCAST PLAYER

Before we deploy to AWS, we need to build both the frontend and backend. In order to build the frontend, run the following inside the **frontend** directory. This should produce a build inside **dist** in the same directory.

```
npm run build
```

The backend consists of three Lambda functions, each built for ARM because it tends to be cheaper. Each component has its own Docker image that builds it, allowing for caching to speed up builds. In order to simplify this process, we've provided a script that enables ARM emulation and builds each of the three components. Run it as shown inside the **backend** directory. This should produce a ZIP file for each component in the **builds** directory.

```
python build_for_deploy.py
```

### 3.3 MANUAL DEPLOYMENT

Now that the frontend and backend have been built, we can deploy using the following command.

```
cdk deploy
```

Sometimes it is helpful to synthesize the Cloudformation template without actually deploying it, in which case `cdk synth` can be used. To show what changes would be made to an existing deployment without actually updating the deployment, run `cdk diff`.

In order to avoid the added cost of a Secrets Manager Endpoint or NAT Gateway in the VPC, the database master password is randomly generated in the CDK and provided to the Migration Function as an environment variable. Once the migrations have run, the Database Service is able to access the database without the credential, since it uses IAM authentication. Generally, it is not a best practice to put the database password in an environment variable, but since the database is already in a VPC not exposed to the internet, and environment variables are not likely to be leaked we've deemed this an acceptable risk. In order to minimize it though, we recommend rotating the database password after deployment by running the rotation script. (Make sure AWS credentials are configured for the script so it is able to run.)

```
python rotate_db_secret.py
```

This script may also be used to rotate the database secret at any time should it be necessary.

### 3.4 CONTINUOUS DEPLOYMENT

Continuous Deployment (CD) allows the Music Podcast Player deployments to be automatically updated as changes are made in GitHub. The workflows we've designed for this purpose automate the process of building the necessary components, deploying the app, and rotating the database secret at the end.

Our CD workflows authenticate with AWS using OpenID Connect. To configure the deployment workflows (workflows that use `deploy.yml`), you will need to set the following environment variables in a GitHub Environment. Several of these variables reference AWS roles that can be assumed by GitHub using OpenID connect when the workflows run. You can set these up following GitHub's documentation on the subject.

1. `AWS_DEPLOY_ROLE`

This is the ARN of an AWS role that can be assumed by GitHub that allows the workflow to assume the following roles.

- (a) `cdk-deploy-role-<account>-<region>`
- (b) `cdk-file-publishing-role-<account>-<region>`
- (c) `cdk-lookup-role-<account>-<region>`
- (d) `cdk-image-publishing-role-<account>-<region>`

2. `AWS_POST_DEPLOY_ROLE`

This is the ARN of an AWS role allowing the workflow to perform the actions defined in post-deploy scripts. Currently, this is just rotating the database credential. The following permissions are required:

- (a) `cloudformation:DescribeStacks`
- (b) `rds:ModifyDBInstance`

3. `AWS_REGION`

This is the AWS region to deploy to.

Additionally, we have a workflow called `cd-checks.yml` which runs on pull requests made to the development branch and ensures that the app is able to build and checks the changes to the current deployment that would be made if the changes were to be deployed. Since it is not actually deploying to AWS, it just needs to be able to assume the `cdk-lookup-role-<account>-<region>` role. If an environment will be running this action, set `AWS_LOOKUP_ROLE` to the ARN of the role the workflow will assume for this.